

Tabela de Mensagens – Firmware STM32G070

v3

Protocolo de quadro:

- **RX** (Raspberry Pi → STM32): `[0xAA] [CMD]` ou `[0xAA] [CMD] [DATA]`
- **TX** (STM32 → Raspberry Pi): `[0x90] [PAYLOAD]` (sucesso), `[0x90] [CMD] [DATA]` (telemetria), ou `[0x91]` (erro)

Tabela 1 – Dicionário Completo de Mensagens

Símbolo	Byte	Direção	Tipo	Descrição
START FRAME _MSG	0xAA	RX	Estrutura	Byte de início de quadro. Todo frame recebido deve começar com este byte; bytes fora de quadro são ignorados silenciosamente.
CMD O K_MSG	0x90	TX	Estrutura	Byte de status positivo. Enviado como primeiro byte de qualquer resposta bem-sucedida, sempre seguido de um payload.
CMD E RR_MS G	0x91	TX	Estrutura	Byte de status negativo. Enviado sozinho (1 byte) quando o comando recebido não é reconhecido.
SYS R DY_MS G	0x10	RX	Handshake	Mensagem enviada pelo Raspberry Pi para iniciar o handshake. Só é aceita após todos os quatro sensores estarem operando na frequência esperada (200 Hz).
SYS I NIT_M SG	0x01	TX	Handshake	Payload de resposta ao handshake. Indica que o STM32 está inicializado. Enviado no quadro <code>[0x90] [0x01]</code> , sempre seguido de <code>[0x90] [0x11]</code> informando o modo inicial.
MODE_ FSM_M SG	0x11	TX	Modo	Payload que informa que o modo ativo é <code>OP_MODE_FSM</code> . Enviado após handshake e após cada transição de modo.
MODE_ DEBUG _MSG	0x22	TX	Modo	Payload que informa que o modo ativo é <code>OP_MODE_DEBUG</code> . Enviado após cada transição de modo.

Símbolo	Byte	Direção	Tipo	Descrição
SW_RESET_MSG	0x33	RX	Controle	Comando de reset por software. Processado com prioridade máxima (antes do toggle de modo e da FSM). O STM32 envia <code>[0x90][0x33]</code> e executa <code>NVIC_SystemReset()</code> após ~50 ms.
ROUTE_A_RECEIVE_MSG	0xDA	RX	FSM	Comando enviado pelo Raspberry Pi ordenando o encaminhamento do objeto para a Rota A. Válido apenas no estado <code>STATE_WAIT_CLASSIFICATION</code> .
ROUTE_B_RECEIVE_MSG	0xDB	RX	FSM	Comando enviado pelo Raspberry Pi ordenando o encaminhamento do objeto para a Rota B. Válido apenas no estado <code>STATE_WAIT_CLASSIFICATION</code> .
OBJECT_DETECTED_MSG	0xA0	TX	FSM	Notificação espontânea de que o Sensor 1 detectou um objeto na entrada da esteira (transição <code>IDLE → OBJECT_DETECTED</code>).
CLASSIFICATION_REQUEST_MSG	0xC0	TX	FSM	Solicitação de classificação enviada ao Raspberry Pi quando o objeto alcança o Sensor 2 (transição <code>OBJECT_DETECTED → WAIT_CLASSIFICATION</code>). O STM32 aguarda <code>0xDA</code> ou <code>0xDB</code> em resposta.
ROUTE_A_FORWARDING_MSG	0xFA	TX	FSM	Confirmação de que o objeto está sendo encaminhado para a Rota A. Enviado ao receber <code>0xDA</code> .
ROUTE_B_FORWARDING_MSG	0xFB	TX	FSM	Confirmação de que o objeto está sendo encaminhado para a Rota B. Enviado ao receber <code>0xDB</code> .
ROUTE_A_SUCCESS_DELIVERY_MSG	0xBA	TX	FSM	Notificação de entrega bem-sucedida na Rota A. Disparado quando o Sensor 3 detecta a passagem do objeto. FSM retorna a <code>STATE_IDLE</code> .
ROUTE_B_SUCCESS_DELIVERY_MSG	0xBB	TX	FSM	Notificação de entrega bem-sucedida na Rota B. Disparado quando o Sensor 4 detecta a passagem do objeto. FSM retorna a <code>STATE_IDLE</code> .
DEBUG_MODE_TOGGLE_MSG	0xDD	RX	Modo	Alterna entre <code>OP_MODE_FSM</code> e <code>OP_MODE_DEBUG</code> . Processado com prioridade máxima (após <code>SW_RESET_MSG</code> , antes da FSM) em qualquer modo de operação.

Símbolo	Byte	Direção	Tipo	Descrição
SENS STATUS _MSG	0x 55	TX	Telemetria	Mensagem espontânea de telemetria dos sensores. Enviada apenas em <code>OP_MODE_DEBUG</code> , somente quando o STATUS_BYTE muda. Frame de 3 bytes: <code>[0x90][0x55][STATUS_BYTE]</code> . Bits 0-3 do STATUS_BYTE indicam o estado de SENS1-SENS4 respectivamente (1 = objeto detectado, 0 = feixe livre). Bits 4-7 reservados (0).
LIGHT EN_M SG	0x E1	RX	Assíncrono (Debug)	Liga o flash/luminária (<code>FLASH_PIN</code> → HIGH). Válido apenas em <code>OP_MODE_DEBUG</code> .
LIGHT DISAB LE_MS G	0x D1	RX	Assíncrono (Debug)	Desliga o flash/luminária (<code>FLASH_PIN</code> → LOW). Válido apenas em <code>OP_MODE_DEBUG</code> .
GATE OPEN_ MSG	0x E2	RX	Assíncrono (Debug)	Abre a cancela movendo o servo para <code>openAngle</code> (45°). Válido apenas em <code>OP_MODE_DEBUG</code> .
GATE CLOSE_ MSG	0x D2	RX	Assíncrono (Debug)	Fecha a cancela movendo o servo para <code>closeAngle</code> (0°). Válido apenas em <code>OP_MODE_DEBUG</code> .
STPR_ EN_MS G	0x E3	RX	Assíncrono (Debug)	Ativa o motor de passo no modo <i>follow steps</i> (eixo engajado, TIM6 habilitado). Válido apenas em <code>OP_MODE_DEBUG</code> .
STPR DISABL E_MSG	0x D3	RX	Assíncrono (Debug)	Desativa o motor de passo com eixo livre (driver A4988 em SLEEP). Válido apenas em <code>OP_MODE_DEBUG</code> .
SET S TPR FO RWARD _MSG	0x E4	RX	Assíncrono (Debug)	Define o sentido de rotação do motor de passo como frente (<code>stepperDirInst</code> = 0). Válido apenas em <code>OP_MODE_DEBUG</code> .
SET S TPR BA CKWARD _MSG	0x D4	RX	Assíncrono (Debug)	Define o sentido de rotação do motor de passo como trás (<code>stepperDirInst</code> = 1). Válido apenas em <code>OP_MODE_DEBUG</code> .
SET S TPR TG T STPS _MSG	0x E5	RX	Assíncrono (Debug)	Inicia um quadro de 3 bytes. O byte <code>DATA</code> subsequente define a quantidade de passos alvo (<code>targetStepps</code>). É o único frame RX de 3 bytes (<code>[0xAA][0xE5][DATA]</code>). Válido apenas em <code>OP_MODE_DEBUG</code> .

Tabela 2 – Fluxo de Mensagens por Evento

Legenda de quadros completos:

- Quadro TX de sucesso (2 bytes): [0x90] [PAYLOAD]
- Quadro TX de telemetria (3 bytes): [0x90] [0x55] [STATUS_BYTE]
- Quadro TX de erro (1 byte): [0x91]
- Quadro RX padrão (2 bytes): [0xAA] [CMD]
- Quadro RX com dado (3 bytes): [0xAA] [0xE5] [DATA]

Modo Normal – OP_MODE_FSM

Sequência de Inicialização (Handshake)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	Sensores prontos + RPi inicia handshake	RPi → STM32	[START] [SYS_RDY]	AA 10	Raspberry Pi sinaliza que está pronto. STM32 só aceita após todos os sensores medirem 200 Hz.
2	STM32 confirma inicialização	STM32 → RPi	[CMD_OK] [SYS_INIT]	90 01	STM32 confirma sistema inicializado.
3	STM32 informa modo inicial	STM32 → RPi	[CMD_OK] [MODE_FSM]	90 11	STM32 informa que inicia em OP_MODE_FSM . Loop de boot encerra. FSM inicia em STATE_IDLE .

Ciclo Completo – Objeto encaminhado para Rota A

#	Evento / Estado FSM	Agente	Quadro Completo	Bytes (hex)	Descrição
1	IDLE → Sensor 1 detecta objeto	STM32 → RPi	[CMD OK] [OBJ DETECTED]	90 A0	STM32 notifica detecção. FSM avança para STATE_OBJECT_DETECTED .
2	OBJECT_DETECTED → Sensor 2 detecta objeto	STM32 → RPi	[CMD OK] [CLSS_REQUEST]	90 C0	STM32 para o motor e solicita classificação. FSM avança para STATE_WAIT_CLASSIFICATION .
3	RPi define destino: Rota A	RPi → STM32	[START] [ROUTE_A]	AA DA	Raspberry Pi envia o resultado da classificação por visão computacional.
4	STM32 confirma e encaminha para Rota A	STM32 → RPi	[CMD OK] [ROUTE_A_FWD]	90 FA	STM32 confirma, aciona servo (closeAngle), religa motor. FSM avança para STATE_ROUTE_A .

#	Evento / Estado FSM	Agente	Quadro Completo	Bytes (hex)	Descrição
5	<code>ROUTE_A</code> → Sensor 3 detecta objeto	STM32 → RPi	[CMD OK] [ROUTE_A_OK]	90 BA	Entrega confirmada na Rota A. FSM retorna para <code>STATE_IDLE</code> .

Ciclo Completo – Objeto encaminhado para Rota B

#	Evento / Estado FSM	Agente	Quadro Completo	Bytes (hex)	Descrição
1	<code>IDLE</code> → Sensor 1 detecta objeto	STM32 → RPi	[CMD OK] [OBJ DETECTED]	90 A0	STM32 notifica detecção. FSM avança para <code>STATE_OBJECT_DETECTED</code> .
2	<code>OBJECT_DETECTED</code> → Sensor 2 detecta objeto	STM32 → RPi	[CMD OK] [CLSS_REQUEST]	90 C0	STM32 para o motor e solicita classificação. FSM avança para <code>STATE_WAIT_CLASSIFICATION</code> .
3	RPi define destino: Rota B	RPi → STM32	[START] [ROUTE_B]	AA DB	Raspberry Pi envia o resultado da classificação por visão computacional.
4	STM32 confirma e encaminha para Rota B	STM32 → RPi	[CMD OK] [ROUTE_B_FWD]	90 FB	STM32 confirma, aciona servo (<code>openAngle</code>), religa motor. FSM avança para <code>STATE_ROUTE_B</code> .
5	<code>ROUTE_B</code> → Sensor 4 detecta objeto	STM32 → RPi	[CMD OK] [ROUTE_B_OK]	90 BB	Entrega confirmada na Rota B. FSM retorna para <code>STATE_IDLE</code> .

Troca de Modo (qualquer estado)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	RPi solicita entrada em modo debug	RPi → STM32	[START] [DEBUG_TOGGLE]	AA DD	Pode ser enviado em qualquer ponto, em qualquer estado da FSM.
2	STM32 ecoa o comando recebido	STM32 → RPi	[CMD OK] [DEBUG_TOGGLE]	90 DD	Confirmação imediata do recebimento do frame.
3	STM32 informa modo ativo	STM32 → RPi	[CMD OK] [MODE_DEBUG]	90 22	Motor travado (<code>stepperStopEngaged</code>), flash apagado. LED pisca 6× rápido.

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
4	RPi solicita retorno ao modo FSM	RPi → STM32	[START] [DEBUG_TOGGLE]	AA DD	Mesmo comando, reenviado para alternar de volta.
5	STM32 ecoa o comando recebido	STM32 → RPi	[CMD OK] [DEBUG_TOGGLE]	90 DD	Confirmação imediata do recebimento do frame.
6	STM32 informa modo ativo	STM32 → RPi	[CMD OK] [MODE_FSM]	90 11	FSM reinicia em <code>STATE_IDLE</code> . LED acende 400 ms e apaga.

Reset por Software (qualquer estado, qualquer modo)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	RPi solicita reset	RPi → STM32	[START] [SW_RESET]	AA 33	Processado com prioridade máxima, antes do toggle de modo e da FSM.
2	STM32 confirma e reinicia	STM32 → RPi	[CMD OK] [SW_RESET]	90 33	STM32 envia confirmação, aguarda ~50 ms para o frame sair pelo TX e executa <code>NVIC_SystemReset()</code> .

Modo Debug – `OP_MODE_DEBUG`

Em modo debug a FSM fica suspensa. O STM32 responde exclusivamente a comandos assíncronos e envia telemetria de sensores espontaneamente.

Telemetria de Sensores (espontânea)

#	Evento	Agente	Quadro Completo	Bytes (hex)	Descrição
1	STATUS_BYTE dos sensores muda	STM32 → RPi	[CMD_OK] [SENS_STATUS] [STATUS_BYTE]	90 55 NN	Enviado apenas quando o valor do STATUS_BYTE muda em relação ao anterior. NN : bit 0 = SENS1, bit 1 = SENS2, bit 2 = SENS3, bit 3 = SENS4 (1 = objeto detectado, 0 = feixe livre). Bits 4–7 reservados (0).

Comandos Assíncronos

#	Comando enviado	Agente	Quadro Completo	Bytes (hex)	Ação executada pelo STM32	Resposta STM32	Bytes (hex)
1	Liga luminária	RPi → STM32	[START] [LIGHT_EN]	AA E1	FLASH_PIN → HIGH	[CMD OK] [LIGHT_EN]	90 E1
2	Desliga luminária	RPi → STM32	[START] [LIGHT_DIS]	AA D1	FLASH_PIN → LOW	[CMD OK] [LIGHT_DIS]	90 D1
3	Abre cancela	RPi → STM32	[START] [GATE_OPEN]	AA E2	Servo → openAngle (45°)	[CMD OK] [GATE_OPEN]	90 E2
4	Fecha cancela	RPi → STM32	[START] [GATE_CLOSE]	AA D2	Servo → closeAngle (0°)	[CMD OK] [GATE_CLOSE]	90 D2
5	Ativa motor (eixo engajado)	RPi → STM32	[START] [STPR_EN]	AA E3	stepperFollowSteps()	[CMD OK] [STPR_EN]	90 E3
6	Desativa motor (eixo livre)	RPi → STM32	[START] [STPR_DIS]	AA D3	stepperStopDisengaged()	[CMD OK] [STPR_DIS]	90 D3
7	Define direção: frente	RPi → STM32	[START] [STPR_FWD]	AA E4	stepperDirInst = 0	[CMD OK] [STPR_FWD]	90 E4
8	Define direção: trás	RPi → STM32	[START] [STPR_BCK]	AA D4	stepperDirInst = 1	[CMD OK] [STPR_BCK]	90 D4
9	Define quantidade de passos	RPi → STM32	[START] [STPR_TGT] [DATA]	AA E5 NN	targetStepps = DATA (NN = valor de 0-255)	[CMD OK] [STPR_TGT]	90 E5
10	Comando desconhecido	RPi → STM32	[START] [???	AA ??	Nenhuma ação	[CMD_ERR]	91